

Welcome back

How was lab? Was anything confusing?

Notes for week 1: <https://tinyurl.com/4ypzsmzu>

Important note: do not read ahead

```
x = [41, 2, 89, 50, 12, 13]
x
```

```
if x[0] % 2 == 0:
    print(x[0])
```

```
if x[1] % 2 == 0:
    print(x[1])
```

```
if x[2] % 2 == 0:
    print(x[2])
```

```
if x[0] % 2 == 0:
    print(x[0])
```

```
if x[1] % 2 == 0:
    print(x[1])
```

```
if x[2] % 2 == 0:
    print(x[2])
```

```
if x[3] % 2 == 0:
    print(x[3])
```

```
if x[4] % 2 == 0:
    print(x[4])
```

```
if x[5] % 2 == 0:
    print(x[5])
```

```
x = [41, 2, 89, 50, 12, 13]
x
```

```
x = [41, 2, 89]
x
```

```

index = 3
if x[index] % 2 == 0:
    print(x[index])

x = [41, 2, 89, 50, 12, 13]
x

index = 0
while index < len(x):
    # How do I tell if x[index] is even?
    if x[index] % 2 == 0:
        print(x[index])
    index += 1
print("Loop finished!")

```

Review

Last time, we covered:

- While loops
- Basic functions

While loop syntax

The syntax for while loops is:

```

while <condition>:
    <expression>

```

As with `if` statements, the `<condition>` can be anything that evaluates to a boolean. The `<expression>` can be complex.

```

a = 1
b = 1
num = 0

while num < 4:
    print(a)
    c = a + b
    a = b
    b = c
    num += 1
    print(a, b, c, num)
    print()

```

```
num = 0
while num < 10:
    print(num)
    if num % 2 == 0:
        print(num)
    num += 1
    print(num)
    print()
```

IMPORTANT: THE WHILE LOOP MUST TERMINATE

```
# WARNING
# WARNING
# WARNING
# DO NOT RUN THIS
```

```
while True:
    print('hi')
```

```
# WARNING
# WARNING
# WARNING
# DO NOT RUN THIS
```

```
num = 0
while num < 10:
    print(num)
```

```
2 ** 8
```

```
summ = 0
index = 1
```

```
summ += index * index
index += 1
```

```
summ += index * index
index += 1
```

```
summ += index * index
index += 1
```

```
summ += index * index
index += 1
```

```
summ += index * index
index += 1
```

```

summ

i = 1
summ = 0
while i <= 5:      # i = 5
    summ += i**2   # summ = 55
    i += 1         # i = 5
print(summ)

# I want to add (and print) the first few numbers
# until their sum exceeds 100.
# but nothing is being printed

total = 0
i = 1
while total <= 100:
    total += i
    i += 1
    print(i)

print("Done!")

```

Indentation

Indentation is critical. It tells Python what code is associated with which Python syntax we've seen so far (if, while, functions).

```

# A tab is four spaces

a = 1
b = 1
if a == 0:      # Level 0
    print(a)    # Level 1
    print("hi") # Level 1

if b == 0:      # Level 0
    print(b)    # Level 1

print(b)        # Level 0

# Do not use one, two, or three spaces.
# Always use four spaces for an indent

a = 0
b = 1
if a == 0:

```

```

    print(a)  # <-- RED IS BAD
    print("hi")
print(b)

# You can "nest" expressions

num = 0          # Level 0 = zero tabs
while num < 5:    # Level 0
    if num % 2 == 0: # Level 1 = one tab
        print(num)  # Level 2 = two tabs
        num += 1    # Level 1

print("Done")    # Level 0

# What's the difference?

num = 0
while num < 5:
    if num % 2 == 0:
        print(num)
        num += 1

print("Done")

```

How Python runs

```

a = 0
a
0

a = 0
a + 1
a
0

a = 0
a + 1
(a + 1) * 2
a
0

num = 0
while num < 5:
    if num % 2 == 0:
        print(num)
        num += 1

```

Printing

You can print anywhere you want. It can help you understand code

```
sum_of_nums = 0
n = 5
index = 1

while index <= n:
    sum_of_nums += index
    index += 1
sum_of_nums

15

sum_of_nums = 0
n = 5
index = 1

while index <= n:
    print("Start of while loop")
    print(sum_of_nums)
    print(index)
    sum_of_nums += index
    index += 1
    print("End of loop")
    print(sum_of_nums)
    print(index)
    print()
sum_of_nums

Start of while loop
0
1
End of loop
1
2

Start of while loop
1
2
End of loop
3
3

Start of while loop
3
3
End of loop
6
```

```
4
Start of while loop
6
4
End of loop
10
5
Start of while loop
10
5
End of loop
15
6
15
```

Conditions

```
x = 1
x == 1
True
x = 1
y = (x == 1)
y
True
if True: # True/False, comparison, variable with a boolean
    print("Hi")
if x == 1:
    print("hi")
if y:
    print("Hi")
Hi
hi
Hi
```

Syntax for functions

The general syntax for functions is:

```
def function_name(<arguments>):  
    <body>  
    return <expression>
```

There are several things to note about the syntax (we will go through examples):

- You can have zero, one, or more arguments.
- You need the `:` after the function
- The body can be complex!
- You technically don't need a return statement at the end, but we will almost always have a return at the end

```
def add_two_numbers(x, y):  
    # x = 5  
    # y = 15  
    s = x + y # s = 20  
    return s
```

```
add_two_numbers(5, 15)
```

```
20
```

```
def add_two_numbers(x, y):  
    # x = 5  
    # y = 15  
    return x
```

```
add_two_numbers(5, 15)
```

```
5
```

```
def add_two_numbers(x, y)  
    return x + y
```

```
add_two_numbers(10, 20)
```

```
Cell In[25], line 1  
    def add_two_numbers(x, y)  
                                ^
```

```
SyntaxError: expected ':'
```

```
def add_two_numbers(x, y):  
    x + y
```

```
add_two_numbers(10, 20)
```

In mathematics, the Fibonacci sequence is a sequence in which each number is the sum of the two preceding ones. Numbers that are part of the Fibonacci sequence are known as Fibonacci numbers, commonly denoted F_n .

$$F_0=0$$

$$F_1=1$$

$$\text{In particular: } F_n = F_{n-1} + F_{n-2}$$

The sequence commonly starts from 0 and 1, although some authors start the sequence from 1 and 1 or sometimes (as did Fibonacci) from 1 and 2. Starting from 0 and 1, the first few values in the sequence are:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144.

```
a = 0 # F_0
b = 1 # F_1
num = 0

c = b + a # F_2 = F_1 + F_0
a = b # F_1
b = c # F_2
num += 1 # 1

c = b + a # F_3 = F_2 + F_1
a = b # F_2
b = c # F_3
num += 1 # 2

c = b + a # F_4 = F_3 + F_2
a = b # F_3
b = c
num += 1 # 3

# F_4 = F_3 + F_2

c

3

def fibonacci(nth):
    # nth = 8
    a = 0
    b = 1
    num = 0

    while num < nth - 1:
        c = a + b
        a = b
        b = c
        num += 1
    return a
```

```
fibonacci(8)
13
fibonacci(15)
377
```

New kind of loop: for loop

In addition to while loops, there is another kind of loop called a `for` loop. Like `while` loops, they allow for repeated computation. The syntax and how they work (semantics) are different though.

As usual we will do examples, the syntax, and go in depth.

```
for x in [1, 2, 3]:
    print(x)

1
2
3
```

Basic syntax for `for` loops

The basic syntax for `for` loops is

```
for <variable> in <iterable expression>:
    <expression> / <body>
```

Don't worry too much about what an `<iterable expression>` is for the time being. For this lecture, it will always be a list, string, dictionary, or `range`. We will cover `range` later.

As with `ifs` and `whiles`, the `<expression>` can be complex.

```
x = [41, 2, 89, 50, 12, 13]
for y in x: # y = x[2]
    print(y)

41
2
89
50
12
13
```

```
x = [41, 2, 89, 50, 12, 13]
```

```
y = x[0]  
print(y)
```

```
y = x[1]  
print(y)
```

```
y = x[2]  
print(y)
```

```
y = x[3]  
print(y)
```

```
y = x[4]  
print(y)
```

```
y = x[5]  
print(y)
```

```
41  
2  
89  
50  
12  
13
```

```
x = [41, 2, 89, 50, 12, 13]  
for y in x:          # Level 0  
    if y % 2 == 0:    # Level 1  
        print(y)      # Level 2
```

```
2  
50  
12
```

```
summ = 0  
x = [1, 2, 3, 4, 5, 6]
```

```
y = x[0]  
summ += y * y
```

```
y = x[1]  
summ += y * y
```

```
y = x[2]  
summ += y * y
```

```
y = x[3]  
summ += y * y
```

```
y = x[4]
summ += y * y
```

```
y = x[5]
summ += y * y
```

```
summ
```

```
91
```

```
x = [1, 2, 3, 4, 5, 6]
sum_of_squares = 0
```

```
for y in x:
    sum_of_squares += y * y
```

```
sum_of_squares
```

```
91
```

```
x = "hello"
```

```
for y in x: # y = x[1]
    print(y)
```

```
h
e
l
l
o
```

```
x = {
    1: 2, # x indexed 0
    3: 4, # x indexed 1
}
```

```
for y in x: # y = x indexed 1
    print(y)
```

```
1
3
```

```
x = {
    1: 2,
    3: 4,
}
```

```
for y in x:
    print(y, x[y])
```

```
1 2
3 4
```

Expressions can be complex!

We've seen conditionals within a for loop. You can also have loops inside loops!

```
for x in [1, 2]:      # Level 0
    for y in [4, 5]: # Level 1
        print(x, y)  # Level 2

for x in [1, 2, 3]:
    y = 0
    while y < x:
        print(y)
        y += 1
    print()
```

Special function: range

When using `for` loops, we will sometimes use a special function called `range`. Don't worry too much about what it returns for the time being - we will stick to using `range` in `for` loops.

```
for x in range(3): # [0, 1, 2]
    print(x)

print()

for x in [0, 1, 2]:
    print(x)

0
1
2

0
1
2

for x in range(10):
    print(x)

0
1
2
3
```

4
5
6
7
8
9

```
for x in range(3, 7): # [3, 4, 5, 6]  
    print(x)
```

3
4
5
6

```
for x in range(6, 9): # [6, 7, 8]  
    print(x)
```

6
7
8

```
for x in range(3, 10, 2): # [3, 5, 7, 9]  
    print(x)
```

3
5
7
9

```
for x in range(10, 3, -1): # [10, 9, 8, 7, 6, 5, 4]  
    print(x)
```

10
9
8
7
6
5
4

```
for x in range(10, 3, -2): # [10, 8, 6, 4]  
    print(x)
```

10
8
6
4

```
for x in range(5.0):  
    print(x)
```

```

-----
-----
TypeError                                Traceback (most recent call
last)
Cell In[78], line 1
----> 1 for x in range(5.0):
      2     print(x)

TypeError: 'float' object cannot be interpreted as an integer

```

One specific use of range

```

x = [47, 6, 9, 2]

for ind in range(len(x)):
    print(ind, x[ind])

# print()
# for y in x:
#     print(y)

0 47
1 6
2 9
3 2

x = [1, 2, 3, 4]
y = [5, 6, 7, 8]

mul_sum = 0
for ind in range(len(x)): # range(4) -> [0, 1, 2, 3]
    mul_sum += x[ind] * y[ind]

mul_sum

for i in range(1, 3, -3):
    print(i)

```

while loops vs for loops

Every `while` loop can be written as a `for` loop and vice versa. When to use which loop depends on the situation at hand. For the time being, we will practice both ways.

```

x = [1, 2, 3, 4]
for y in x:
    print(y * y)

```

```

print()

i = 0
while i < len(x):
    print(x[i] * x[i])
    i += 1

1
4
9
16

1
4
9
16

for num in range(10, 30, 7):
    print(num)

print()

num = 10
while num < 30:
    print(num)
    num += 7

10
17
24

10
17
24

```

Advanced loop concepts: break

Sometimes we want to end a loop early. We can do this using the `break` keyword.

```

x = [1, 4, 7, 9, 11, 15]

# Compute the squared sum of the values of x until we reach 7
squared_sum = 0
for y in x: # y = x[2]
    if y == 7:
        break
    squared_sum += y * y

```

```

squared_sum
17
# Fun exercise: do the same with a while loop!
# Breaks can come anywhere in the body of a for loop

y = [1, 2, 3]
for x in y: # x = y[0]
    break
    print(x)

print("hi")
hi

```

Advanced loop concepts: continue

Sometimes we want to skip the rest of the body of a loop. We can do this using the `continue` keyword.

```

x = [1, 4, 7, 9, 11, 15]

# Compute the squared sum of the _even_ values of x
squared_sum = 0
for y in x: # y = x[3]
    if y % 2 == 1:
        continue
    squared_sum += y * y

squared_sum
x = [1, 4, 7, 9, 11, 15]

# Compute the squared sum of the _odd_ values of x
squared_sum = 0
for y in x:
    if y % 2 == 0:
        continue
    squared_sum += y * y

squared_sum
x = [1, 4, 7, 9, 11, 15]

# Compute the squared sum of the _odd_ values of x. Stop at 9
squared_sum = 0

```

```
for y in x: # y = x[3]
    if y % 2 == 0:
        continue
    if y == 9:
        break
    squared_sum += y * y

squared_sum
```

Review redux: syntax for functions

The general syntax for functions is:

```
def function_name(<arguments>):
    <body>
    return <expression>
```

There are several things to note about the syntax (we will go through examples):

- You can have zero, one, or more arguments.
- You need the `:` after the function
- The body can be complex!
- You technically don't need a return statement at the end, but we will almost always have a return at the end

```
def fahrenheit_to_celsius(f):
    # ???
    return 0

def celsius_to_fahrenheit(f):
    # ???
    return 0
```

A more involved example

Let's say we had a meal with friends. We ordered multiple items.

Let's put the price of the items with a list:

- One shiro: 100 birr
- One tibbs: 200 birr
- Two firr-firr: 150 birr x 2

The list might look like `[100, 200, 150, 150]`

Let's create a function that computes the total for the bill.

```
def sum_bill(bill):
    total = 0
    for item in bill:
        total += item
    return total

sum_bill([100, 200, 150, 150])

sum_bill([100, 200, 200, 150, 150])
```

But wait! We forgot the tax. Let's say there's a 5% tax rate. We can *reuse* functions we've created before. Let's try that now

```
def total_after_tax(bill):
    sum_total = sum_bill(bill)
    return sum_total * 1.05

total_after_tax([100, 200, 150, 150])
```

Functions can call functions!

As we've seen, functions can call functions. This will become critical later!

Warning: function naming

Functions cannot have the same name as a variable and vice versa. For this class, do not name a function a variable you've already declared and vice versa.

More on function and variable names

There are specific *reserved keywords* in Python. Do not name functions and variables these reserved keywords:

Types:

- `int`
- `float`
- `str`
- `bool`
- `list`
- `dict`

Syntax:

- `if`
- `while`
- `for`
- `break`
- `continue`

```
if = 1
```